Class 13: RNASeq with DESeq2

Saket Chodavarapu (PID: A18582086)

Table of contents

Background . . . 1
Data Import . . . 1
Check on metadata counts correspondance . . . 2
Analysis Plan… . . . 3
Log2 units and fold change . . . 6
Remove zero count genes . . . 7
DESeq analysis . . . 8
Volcano plot . . . 10
Add some plot annotation . . . 11
Save our results to a CSV file . . . 12
Add Annotation Data . . . 12
Pathway analysis . . . 14

Background

TOday we will perform an RNASeq analysis on the effects of dexamethasone (hereafter referred to as "dex"), a common steroid, on airway smooth muscle (ASM) cell lines.

Data Import

We need two things for this analysis:

- **countData**: a table with genes as rows and samples/experiments as columns, = **colData**: metadata about the columns (i.e. samples) in the main countData object.

```
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
```

Let's have a wee peek at these two objects:

```r
metadata
```

```
          id     dex celltype     geo_id
1 SRR1039508 control   N61311 GSM1275862
2 SRR1039509 treated   N61311 GSM1275863
3 SRR1039512 control  N052611 GSM1275866
4 SRR1039513 treated  N052611 GSM1275867
5 SRR1039516 control  N080611 GSM1275870
6 SRR1039517 treated  N080611 GSM1275871
7 SRR1039520 control  N061011 GSM1275874
8 SRR1039521 treated  N061011 GSM1275875
```

```r
head(counts)
```

```
                SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516
ENSG00000000003        723        486        904        445       1170
ENSG00000000005          0          0          0          0          0
ENSG00000000419        467        523        616        371        582
ENSG00000000457        347        258        364        237        318
ENSG00000000460         96         81         73         66        118
ENSG00000000938          0          0          1          0          2
                SRR1039517 SRR1039520 SRR1039521
ENSG00000000003       1097        806        604
ENSG00000000005          0          0          0
ENSG00000000419        781        417        509
ENSG00000000457        447        330        324
ENSG00000000460         94        102         74
ENSG00000000938          0          0          0
```

Check on metadata counts correspondance

We need to check that the metadata matches the samples in our count data.

```r
ncol(counts) == nrow(metadata)
```

```
[1] TRUE
```

```r
colnames(counts) == metadata$id
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```r
all(colnames(counts) == metadata$id)
```

```
[1] TRUE
```

> Q1. How many genes are in this dataset?

```r
nrow(counts)
```

```
[1] 38694
```

There are 38694 genes in this dataset.

> Q2. How many "control" samples are in this dataset?

```r
sum(metadata$dex=="control")
```

```
[1] 4
```

There are 4 control samples in this dataset.

Analysis Plan...

We have 4 replicates per condition ("control" and "treated"). We want to compare the control vs the treated to see which genes' expression levels change when we have the drug present.

We will go row by row (each gene) to see if the average value in control columns is different than the average value in treated columns.

- Step 1. Find which columns in `counts` correspond to "control" samples.
- Step 2. Extract/select these columns.
- Step 3. Calculate an average value for each gene (i.e. each row).

```r
# The indices (i.e. positions) that are "control"
control.inds <- metadata$dex == "control"
```

```r
# Extract/select these "control" columns from counts
control.counts <- counts[, control.inds]
```

```r
# Calculate the mean for each gene (i.e. row)
control.mean <- rowMeans(control.counts)
```

> Q3. How would you make the above code in either approach more robust? Is there a function that could help here?

Using the `rowMeans()` function like we did above is a more robust approach than just using `rowSums()` and dividing by an arbitrary value.

> Q4. Do the same for "treated" samples - find the mean count value per gene.

```r
treated.mean <- rowMeans( counts[, metadata$dex == "treated"] )
```

Let's put these two mean values into a new data.frame `meancounts` for easy book-keeping and plotting.

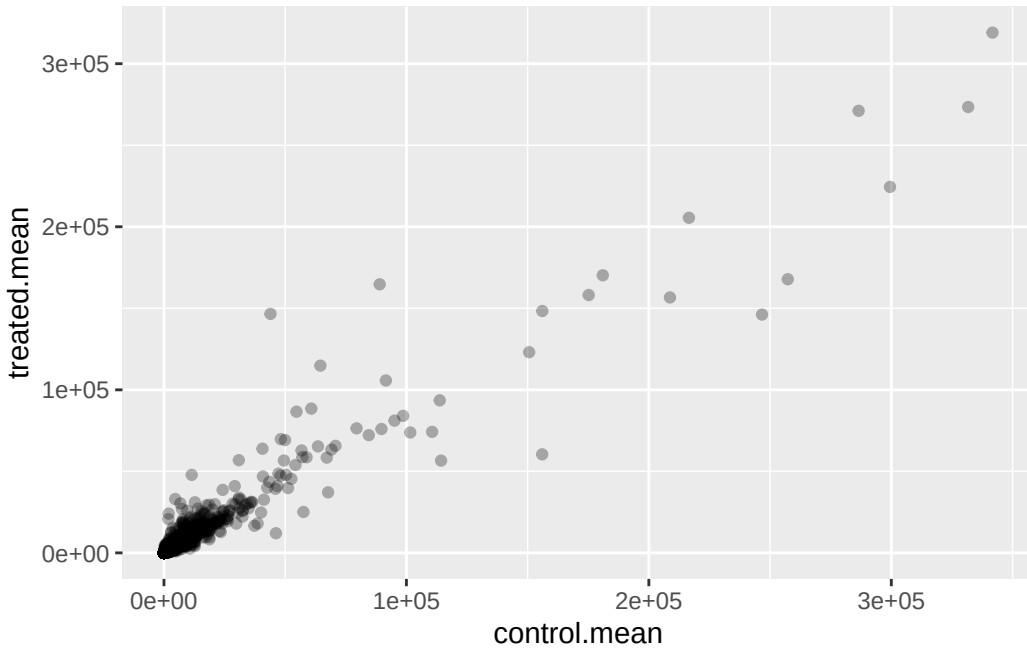
```r
meancounts <- data.frame(control.mean, treated.mean)
head(meancounts)
```

```
                control.mean treated.mean
ENSG00000000003       900.75       658.00
ENSG00000000005         0.00         0.00
ENSG00000000419       520.50       546.00
ENSG00000000457       339.75       316.50
ENSG00000000460        97.25        78.75
ENSG00000000938         0.75         0.00
```

> Q. Make a ggplot of average counts of control vs treated.

```r
library(ggplot2)

ggplot(meancounts) +
  aes(x=control.mean, y=treated.mean) +
  geom_point(alpha=0.3)
```

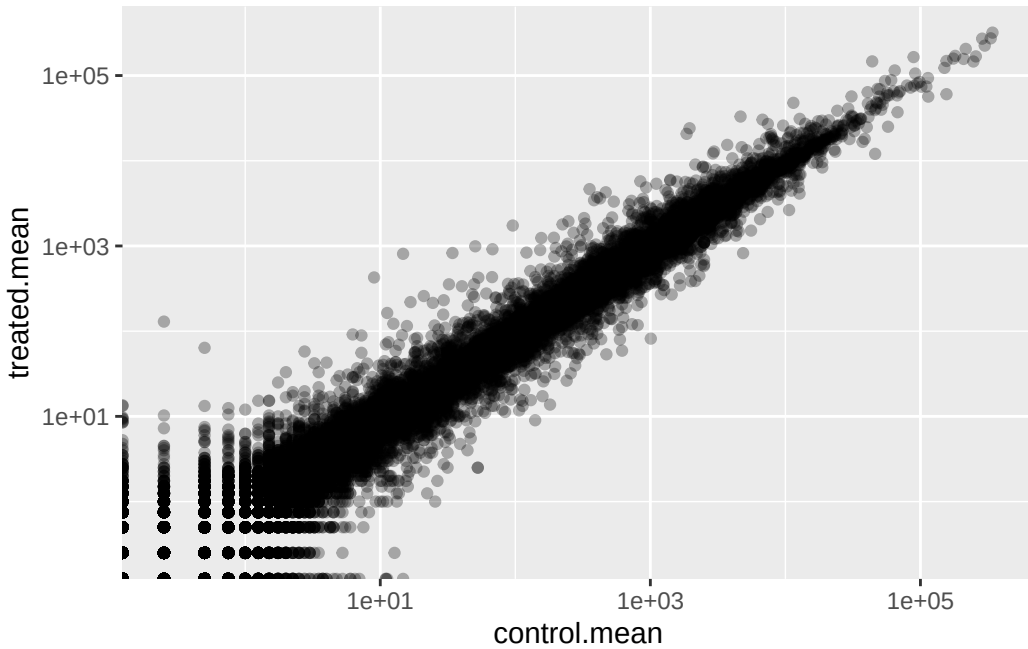


This is screaming to be log transformed as it is so highly skewed…

```
ggplot(meancounts) +
  aes(x=control.mean, y=treated.mean) +
  geom_point(alpha=0.3) +
  scale_x_log10() +
  scale_y_log10()
```

```
Warning in scale_x_log10(): log-10 transformation introduced infinite values.

Warning in scale_y_log10(): log-10 transformation introduced infinite values.
```



Log2 units and fold change

If we consider "treated"/"control" counts we will get a number that tells us the change.

```
# A doubling in the treated vs control
log2(40/20)
```

```
[1] 1
```

```
log2(10/20)
```

```
[1] -1
```

> Q. Add a new column `log2fc` for log2 fold change of treated/control to our `meancounts` object.

```
meancounts$log2fc <-
  log2(meancounts$treated.mean / meancounts$control.mean)

head(meancounts)
```

```
                 control.mean treated.mean      log2fc
ENSG00000000003       900.75       658.00 -0.45303916
ENSG00000000005         0.00         0.00         NaN
ENSG00000000419       520.50       546.00  0.06900279
ENSG00000000457       339.75       316.50 -0.10226805
ENSG00000000460        97.25        78.75 -0.30441833
ENSG00000000938         0.75         0.00        -Inf
```

Remove zero count genes

Typically we would not consider zero count genes - as we have no data about them and they should be excluded from further consideration. These lead to "funky" log2 fold change values (e.g. divide by zero errors etc.)

```
zero.vals <- which(meancounts[,1:2]==0, arr.ind=TRUE)

to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
head(mycounts)
```

```
                 control.mean treated.mean      log2fc
ENSG00000000003       900.75       658.00 -0.45303916
ENSG00000000419       520.50       546.00  0.06900279
ENSG00000000457       339.75       316.50 -0.10226805
ENSG00000000460        97.25        78.75 -0.30441833
ENSG00000000971      5219.00      6687.50  0.35769358
ENSG00000001036      2327.00      1785.75 -0.38194109
```

> Q7. What is the purpose of the `arr.ind` argument in the **which()** function call above? Why would we then take the first column of the output and need to call the **unique()** function?

The `arr.ind` argument tells us which genes and samples have zero counts. We will look at the rows with zero counts.

```
up.ind <- mycounts$log2fc > 2
down.ind <- mycounts$log2fc < (-2)
```

```
sum(up.ind)
```

```
[1] 250
```

```r
sum(down.ind)
```

```
[1] 367
```

> Q8. Using the `up.ind` vector above can you determine how many up regulated genes we have at the greater than 2 fc level?

250 genes

> Q9. Q9. Using the `down.ind` vector above can you determine how many down regulated genes we have at the greater than 2 fc level?

367 genes

> Q10. Do you trust these results? Why or why not?

We do not know if the fold change value is statistically significant, so I do not trust these results.

DESeq analysis

We are missing any measure of significance from the work we have so far. Let's do this properly with the **DESeq2** package.

```r
library(DESeq2)
```

The DESeq2 package, like many bioconductor packages, wants its input in a very specific way - a data structure setup with all the info it needs for the calculation.

```r
dds <- DESeqDataSetFromMatrix(countData = counts,
                              colData = metadata,
                              design = ~dex)
```

```
converting counts to integer mode
```

```
Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
design formula are characters, converting to factors
```

The main function in this package is called `DESeq()` it will run the full analysis for us on our `dds` input object:

```r
dds <- DESeq(dds)
```

```
estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing
```

```r
res <- results(dds)
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 6 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195      -0.350703  0.168242 -2.084514 0.0371134
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160       0.206107  0.101042  2.039828 0.0413675
ENSG00000000457 322.664844       0.024527  0.145134  0.168996 0.8658000
ENSG00000000460  87.682625      -0.147143  0.256995 -0.572550 0.5669497
ENSG00000000938   0.319167      -1.732289  3.493601 -0.495846 0.6200029
                     padj
                <numeric>
ENSG00000000003  0.163017
ENSG00000000005        NA
ENSG00000000419  0.175937
ENSG00000000457  0.961682
ENSG00000000460  0.815805
ENSG00000000938        NA
```

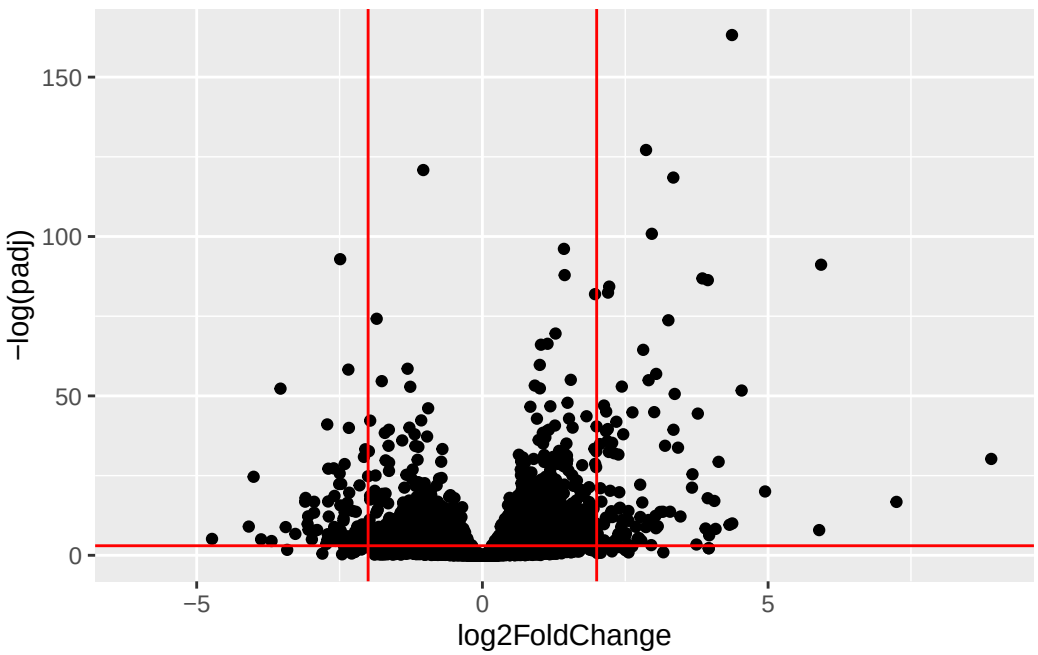
Volcano plot

A useful summary figure of our results is often called a volcano plot. It is basically a plot of log2 fold change values vs Adjusted P-values.

> Q. Use ggplot to make a first version "volcano plot" of `log2FoldChange` vs `padj`.

```
ggplot(res) +
  aes(log2FoldChange, -log(padj)) +
  geom_point() +
  geom_vline(xintercept = c(-2,+2), col="red") +
  geom_hline(yintercept = -log(0.05), col="red")
```

```
Warning: Removed 23549 rows containing missing values or values outside the scale range
(`geom_point()`).
```



This is not very useful because the y-axis (P-value) is not really helpful - we want to focus on low P-values.

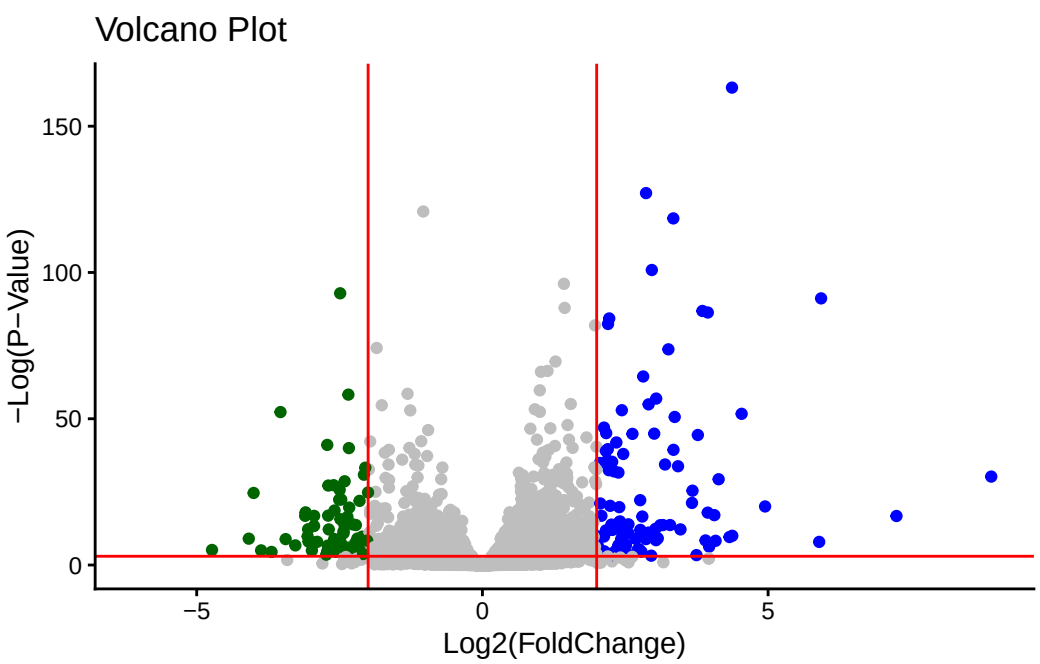
Add some plot annotation

> Q. Add color to the points (genes) we care about, nice axis labels, a useful title, and a nice theme.

```
mycols <- rep("gray", nrow(res))
mycols[ res$log2FoldChange > 2 ] = "blue"
mycols[ res$log2FoldChange < -2 ] = "darkgreen"
mycols[ res$padj >= 0.05 ] = "gray"
```

```
ggplot(res) +
  aes(log2FoldChange, -log(padj)) +
  labs(title="Volcano Plot", x="Log2(FoldChange)", y="-Log(P-Value)") +
  geom_point(col=mycols) +
  geom_vline(xintercept = c(-2,+2), col="red") +
  geom_hline(yintercept = -log(0.05), col="red") +
  theme_classic()
```

```
Warning: Removed 23549 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Save our results to a CSV file

```
write.csv(res, file="results.csv")
```

Add Annotation Data

To make sense of our results we need to know what the differentially expressed genes are and what biological pathways and process they are involved in.

Let's start by mapping our ENSEMBLE ids to the more conventional gene SYMBOL.

We will use two bioconductor packages for this "mapping" **AnnotationDbi** and **org.Hs.eg.db**.

We will first need to install these from bioconductor with `BiocManager::install("AnnodationDbi")`

```
library(AnnotationDbi)
library(org.Hs.eg.db)
```

```
columns(org.Hs.eg.db)
```

```
 [1] "ACCNUM"       "ALIAS"        "ENSEMBL"      "ENSEMBLPROT"  "ENSEMBLTRANS"
 [6] "ENTREZID"     "ENZYME"       "EVIDENCE"     "EVIDENCEALL"  "GENENAME"
[11] "GENETYPE"     "GO"           "GOALL"        "IPI"          "MAP"
[16] "OMIM"         "ONTOLOGY"     "ONTOLOGYALL"  "PATH"         "PFAM"
[21] "PMID"         "PROSITE"      "REFSEQ"       "SYMBOL"       "UCSCKG"
[26] "UNIPROT"
```

```
res$symbol <- mapIds(org.Hs.eg.db,
                     keys = rownames(res), # Our ids
                     keytype = "ENSEMBL", # Their format
                     column = "SYMBOL") # What I want to translate to
```

```
'select()' returned 1:many mapping between keys and columns
```

```
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
DataFrame with 6 rows and 7 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195      -0.350703  0.168242 -2.084514 0.0371134
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160       0.206107  0.101042  2.039828 0.0413675
ENSG00000000457 322.664844       0.024527  0.145134  0.168996 0.8658000
ENSG00000000460  87.682625      -0.147143  0.256995 -0.572550 0.5669497
ENSG00000000938   0.319167      -1.732289  3.493601 -0.495846 0.6200029
                     padj      symbol
                <numeric> <character>
ENSG00000000003  0.163017      TSPAN6
ENSG00000000005        NA        TNMD
ENSG00000000419  0.175937        DPM1
ENSG00000000457  0.961682       SCYL3
ENSG00000000460  0.815805       FIRRM
ENSG00000000938        NA         FGR
```

Q. Can you add "GENENAME" and "ENTREZID" as new columns to `res` named "name" and "entrez"?

```r
res$name <- mapIds(org.Hs.eg.db,
                   keys = rownames(res),
                   keytype = "ENSEMBL",
                   column = "GENENAME")
```

```
'select()' returned 1:many mapping between keys and columns
```

```r
res$entrez <- mapIds(org.Hs.eg.db,
                   keys = rownames(res),
                   keytype = "ENSEMBL",
                   column = "ENTREZID")
```

```
'select()' returned 1:many mapping between keys and columns
```

```r
head(res)
```

```
log2 fold change (MLE): dex treated vs control
Wald test p-value: dex treated vs control
```

```
DataFrame with 6 rows and 9 columns
                  baseMean log2FoldChange     lfcSE      stat    pvalue
                 <numeric>      <numeric> <numeric> <numeric> <numeric>
ENSG00000000003 747.194195      -0.350703  0.168242 -2.084514 0.0371134
ENSG00000000005   0.000000             NA        NA        NA        NA
ENSG00000000419 520.134160       0.206107  0.101042  2.039828 0.0413675
ENSG00000000457 322.664844       0.024527  0.145134  0.168996 0.8658000
ENSG00000000460  87.682625      -0.147143  0.256995 -0.572550 0.5669497
ENSG00000000938   0.319167      -1.732289  3.493601 -0.495846 0.6200029
                     padj      symbol                   name      entrez
                 <numeric> <character>            <character> <character>
ENSG00000000003  0.163017      TSPAN6          tetraspanin 6        7105
ENSG00000000005        NA        TNMD            tenomodulin       64102
ENSG00000000419  0.175937        DPM1 dolichyl-phosphate m..        8813
ENSG00000000457  0.961682       SCYL3 SCY1 like pseudokina..       57147
ENSG00000000460  0.815805       FIRRM FIGNL1 interacting r..       55732
ENSG00000000938        NA         FGR FGR proto-oncogene, ..        2268
```

```
write.csv(res, file="results_annotated.csv")
```

Pathway analysis

Now we know the gene names (gene symbols) and their entrez IDs we can find out what pathways they are involved in. This is called "pathway analysis" or "gene set enrichment"

We will use the **gage** package and the **pathview** package to do this analysis (but there are loads of others).

```
library(gage)
library(gageData)
library(pathview)
```

Let's see what is in gageData, specifically KEGG pathways:

```
data(kegg.sets.hs)
head(kegg.sets.hs, 2)
```

```
$`hsa00232 Caffeine metabolism`
[1] "10"   "1544" "1548" "1549" "1553" "7498" "9"

$`hsa00983 Drug metabolism - other enzymes`
```

```
 [1] "10"     "1066"   "10720"  "10941"  "151531" "1548"   "1549"   "1551"
 [9] "1553"   "1576"   "1577"   "1806"   "1807"   "1890"   "221223" "2990"
[17] "3251"   "3614"   "3615"   "3704"   "51733"  "54490"  "54575"  "54576"
[25] "54577"  "54578"  "54579"  "54600"  "54657"  "54658"  "54659"  "54963"
[33] "574537" "64816"  "7083"   "7084"   "7172"   "7363"   "7364"   "7365"
[41] "7366"   "7367"   "7371"   "7372"   "7378"   "7498"   "79799"  "83549"
[49] "8824"   "8833"   "9"      "978"
```

To run our pathway analysis we will use the **gage()** function. It wants to main inputs: a vector of importance (in our case the log2 fold change values); and the gene set to check overlap for.

```r
foldchanges <- res$log2FoldChange
names(foldchanges) <- res$symbol
head(foldchanges)
```

```
     TSPAN6        TNMD        DPM1       SCYL3       FIRRM         FGR
-0.35070296          NA  0.20610728  0.02452701 -0.14714263 -1.73228897
```

KEGG speaks entrez (i.e. uses ENTREZid format) not gene symbol format

```r
names(foldchanges) <- res$entrez
```

```r
keggres <- gage(foldchanges, gsets=kegg.sets.hs)
```

```r
head(keggres$less, 5)
```

```
                                                      p.geomean stat.mean
hsa05332 Graft-versus-host disease                   0.0004250607 -3.473335
hsa04940 Type I diabetes mellitus                    0.0017820379 -3.002350
hsa05310 Asthma                                      0.0020046180 -3.009045
hsa04672 Intestinal immune network for IgA production 0.0060434609 -2.560546
hsa05330 Allograft rejection                         0.0073679547 -2.501416
                                                             p.val      q.val
hsa05332 Graft-versus-host disease                   0.0004250607 0.09053792
hsa04940 Type I diabetes mellitus                    0.0017820379 0.14232788
hsa05310 Asthma                                      0.0020046180 0.14232788
hsa04672 Intestinal immune network for IgA production 0.0060434609 0.31387487
hsa05330 Allograft rejection                         0.0073679547 0.31387487
                                                     set.size         exp1
hsa05332 Graft-versus-host disease                         40 0.0004250607
```

```
hsa04940 Type I diabetes mellitus                          42 0.0017820379
hsa05310 Asthma                                            29 0.0020046180
hsa04672 Intestinal immune network for IgA production      47 0.0060434609
hsa05330 Allograft rejection                               36 0.0073679547
```

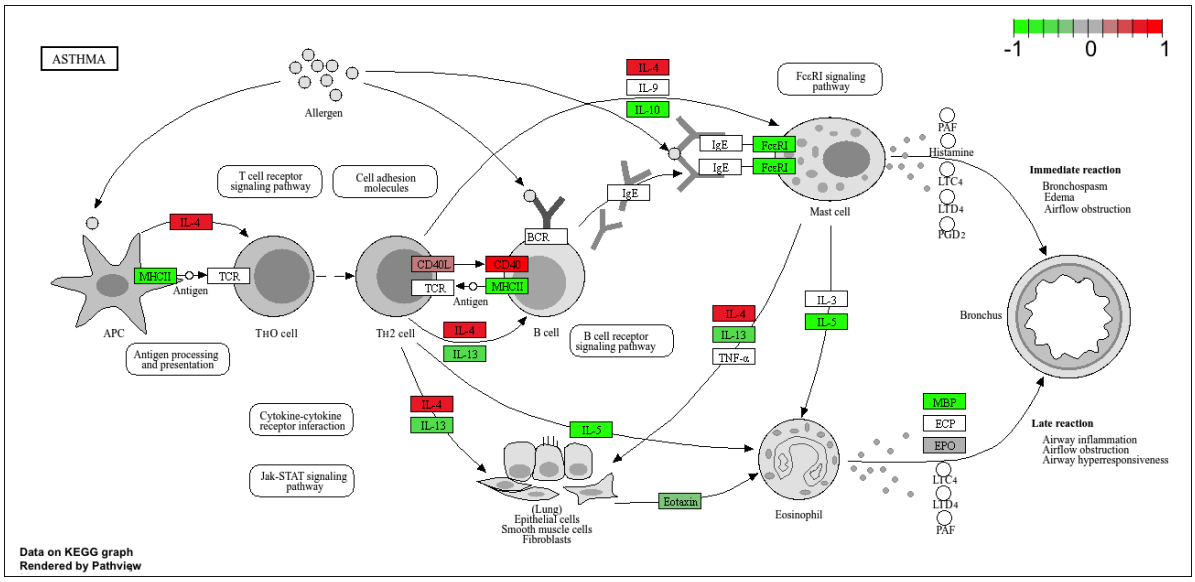
Let's make a figure of one of these pathways with our DEGs highlighted:

```
pathview(foldchanges, pathway.id = "hsa05310")
```

```
'select()' returned 1:1 mapping between keys and columns
```

```
Info: Working in directory /Users/saketchodavarapu/Documents/BIMM143/Class13Feb18
```

```
Info: Writing image file hsa05310.pathview.png
```



Q. Generate and insert a pathway figure for "Graft-versus-host disease" and "Type I diabetes"?

```
pathview(foldchanges, pathway.id = "hsa05332")
```

```
'select()' returned 1:1 mapping between keys and columns
```

```
Info: Working in directory /Users/saketchodavarapu/Documents/BIMM143/Class13Feb18
```

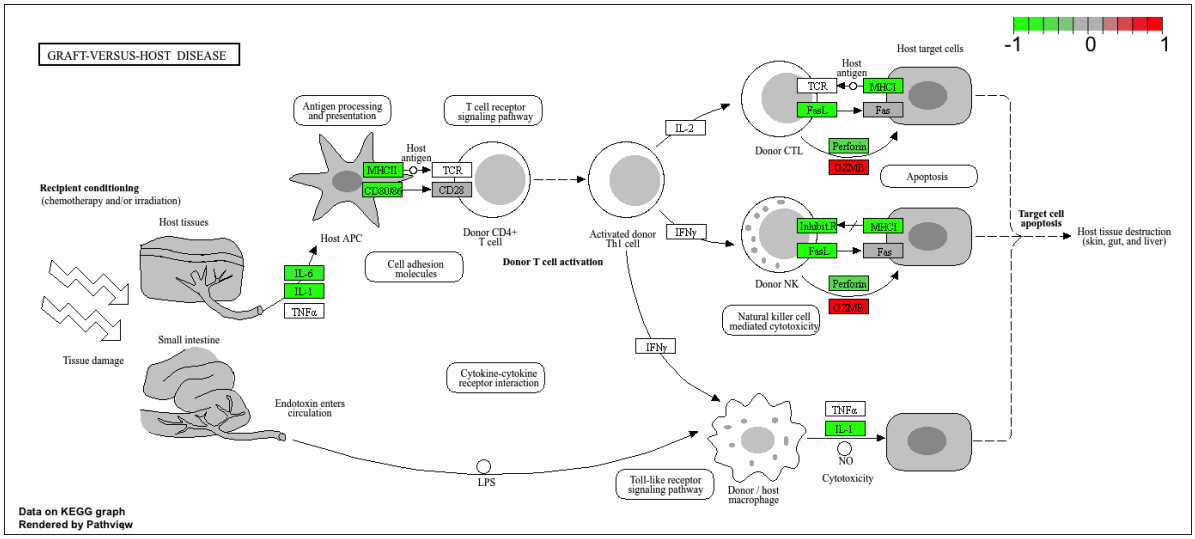
```
Info: Writing image file hsa05332.pathview.png
```

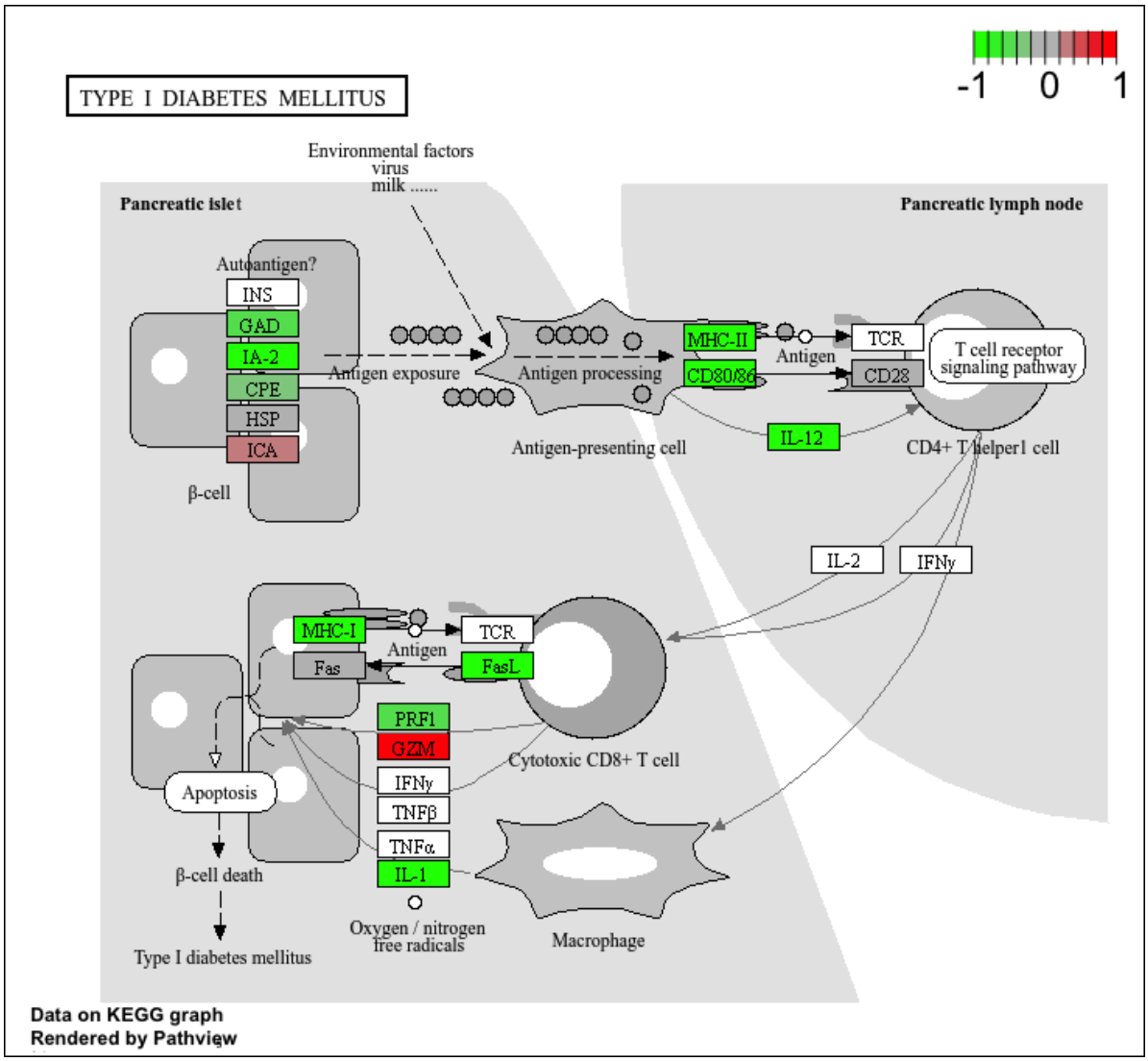
```r
pathview(foldchanges, pathway.id = "hsa04940")
```

```
'select()' returned 1:1 mapping between keys and columns

Info: Working in directory /Users/saketchodavarapu/Documents/BIMM143/Class13Feb18

Info: Writing image file hsa04940.pathview.png
```




TYPE I DIABETES MELLITUS
-1
0
1
Environmental factors
virus
milk .....
Pancreatic islet
Pancreatic lymph node
Autoantigen?
INS
GAD
IA-2
CPE
HSP
ICA
β-cell
Antigen exposure
Antigen processing
Antigen-presenting cell
MHC-II
CD80/86
Antigen
TCR
CD28
T cell receptor signaling pathway
IL-12
CD4+ T helper1 cell
IL-2
IFNγ
MHC-I
Fas
Antigen
TCR
FasL
PRF1
GZM
IFNγ
TNFβ
TNFα
IL-1
Cytotoxic CD8+ T cell
Apoptosis
β-cell death
Type I diabetes mellitus
Oxygen / nitrogen
free radicals
Macrophage
Data on KEGG graph
Rendered by Pathview